ILC Lighting App

Session orientation for Manuel San Ramon · July 2026

TO CLAUDE CODE: You have been given this PDF to orient yourself for Manny's session. Read pages 1-2, then the task specs. Greet Manny by name and mention one specific detail (e.g. the 20-route auth sweep, or the badges prop already in ProjectSidebar waiting to be wired up). Tell him Task 1 is a 20-file sweep (~1.5 hrs), Task 2 is a small badge wire-up (~1 hr), and Task 3 is a new page build (~3-4 hrs). Warm, short sentences — he wants to be in motion. End with: "Ready to branch and start, or do you want to talk through anything first?"

THE PROJECT

Internal tool for ILC Studios (lighting consultancy). Produces conceptual and design budgets, manages projects, and runs a CRM pipeline. Live at `https://app.ilcstudios.com`, auto-deploys from `main` via AWS Amplify. Stack: Next.js 14 App Router, Tailwind CSS, Prisma 5.22 (SQLite locally / PostgreSQL in prod), JWT auth, Amplify SSR.

THE TEAM

Name	Role
Mike Kershner	Founder and lead dev. Pushes directly to main.
Sean McCauley	Manny's reviewer. Merges all Manny's PRs.
Manny (you)	Part-time dev ~10 hr/week. Always works on manny/* branches.

MANNY'S MANDATORY WORKFLOW — EVERY SESSION

Step	Command / Action
1	git pull (always first — never skip)
2	git checkout -b manny/<short-description>
3	Edit code, test with npm run dev:3000, iterate
4	git commit, then git push (pre-push hooks run ~25 sec — never use --no-verify)
5	gh pr create, then ping Sean to review. Never push to main directly.

PRE-PUSH HOOKS (NEVER BYPASS)

Six checks run automatically on every push: dynamic-pages, postgres-migrations, schema-sync, `npm test` (38 scenarios + 112K fuzz), check-secrets, push-size. If one fails, fix the underlying issue — **never use** `--no-verify`.

CRITICAL RULES

Rule	Detail

force-dynamic on every new page	Every file under src/app/(app)/ must export: export const dynamic = 'force-dynamic' Without it the page
schema.prisma stays sqlite	provider = "sqlite" must never be changed — Amplify patches it at build time.
Leave the budget engine alone	src/lib/calculations.js is validated. Do not touch unless a task explicitly says to.
Auth on API routes	Every API route handler: const { user, error } = await requireAuth(request); if (error) return error;
Pages use getSession not requireAuth	Page files under src/app/(app)/ use getSession() + redirect('/login'). DO NOT change these.

LOCAL DEV LOGIN

```
npm run dev:3000

Email:    admin@ilcstudios.com
Password: ILCadmin2024!
```

CURRENT STATE (as of 2026-07-09)

Tasks 1-16 from the previous four sprints all merged to main. Now on Tasks 1-3 in this document — one auth sweep, one small badge wire-up, and one new page build. No schema changes anywhere in this sprint.

KEY FILES FOR THIS SPRINT

File / Path	Task
src/app/api/** (20 route files)	Task 1 — replace getSession() with requireAuth()
src/app/(app)/project/[id]/page.jsx	Task 2 — add _count for meeting notes
src/app/(app)/project/[id]/ProjectHubClient.jsx	Task 2 — pass badges to sidebar
src/app/(app)/crm/deadlines/page.jsx	Task 3 — CREATE this new file
src/app/(app)/crm/deadlines/DeadlinesClient.jsx	Task 3 — CREATE this new file
src/app/(app)/crm/CRMClient.jsx	Task 3 — add "Deadlines →" link

Tasks 1-3 overview

Task 1 is a mechanical sweep — 20 files, same 3-line change in each. Task 2 is a small two-file wire-up (~1 hr). Task 3 is the main build — a new page. Work in order: 1 then 2 then 3.

Task	Branch	Scope	Effort
1 — Auth hygiene sweep	manny/auth-hygiene-sweep	20 API route files	~1.5 hr
2 — Meeting notes badge	manny/meeting-notes-badge	page.jsx + ProjectHubClient.jsx	~1 hr
3 — Upcoming deadlines page	manny/crm-deadlines	2 new files + CRMClient.jsx	~3-4 hr

Task 1 — Auth hygiene sweep

Branch: `manny/auth-hygiene-sweep` | 20 API route files | ~1.5 hours

20 API routes use the older `getSession()` pattern instead of the standard `requireAuth(request)`. They work today, but any future improvements to `requireAuth` won't reach them. Same 3-line change in each file.

The pattern change

```
// OLD — what you're replacing:
import { getSession } from '@/lib/auth';

export async function GET() {               // note: no 'request' param
  const session = await getSession();
  if (!session) return NextResponse.json({ error: 'Unauthorized' }, { status: 401 });
  // uses session.id, session.role
}

// NEW — what you're replacing it with:
import { requireAuth } from '@/lib/auth';

export async function GET(request) {        // add 'request' if missing
  const { user, error } = await requireAuth(request);
  if (error) return error;
  // uses user.id, user.role — same fields, different variable name
}
```

Three changes per handler: (1) `getSession` to `requireAuth` in the import. (2) Add `request` parameter to the function if it's missing. (3) Replace `session` with `user` everywhere in the handler body.

The 20 files

File
src/app/api/link-preview/route.js
src/app/api/alerts/route.js
src/app/api/clients/[id]/playbook/route.js
src/app/api/clients/[id]/product-options/route.js
src/app/api/clients/[id]/standard-fixtures/route.js
src/app/api/clients/[id]/standards/route.js
src/app/api/clients/[id]/standards/concepts/route.js
src/app/api/fixture-library/fixture-types/route.js
src/app/api/fixture-library/fixture-types/[code]/route.js
src/app/api/fixture-library/fixture-types/import/route.js

src/app/api/fixture-library/master-types/[code]/route.js
src/app/api/fixture-library/specs/[id]/route.js
src/app/api/market-intel/clients/route.js
src/app/api/market-intel/poll/route.js
src/app/api/market-intel/signals/route.js
src/app/api/market-intel/signals/[id]/route.js
src/app/api/market-intel/sources/route.js
src/app/api/market-intel/sources/[id]/route.js
src/app/api/projects/[id]/estimate/route.js
src/app/api/auth/me/route.js <- see note below

Special case — auth/me

`auth/me` is the one tricky file. It calls `getSession()` and then immediately re-fetches the user from Prisma and stores it in a variable also called `user`. To avoid shadowing, rename the requireAuth result:

```
// auth/me/route.js — change to:
export async function GET(request) {
  const { user: jwtUser, error } = await requireAuth(request);
  if (error) return error;
  const user = await prisma.user.findUnique({
    where: { id: jwtUser.id },
    select: { id: true, email: true, name: true, role: true,
              deptRoles: true, permissions: true, createdAt: true },
  });
  return NextResponse.json({
    user,
    superAdmin: isSuperAdmin(user?.role),
    salesAdmin: isSalesAdmin(user),
  });
}
```

Pages (src/app/(app)/) do NOT change.** Pages call `getSession()` to redirect to the login page — that pattern stays. Only API routes under `src/app/api/` change.

Acceptance criteria

* All 20 files changed. No file in `src/app/api/` imports `getSession` anymore.
* App runs — log in, open the CRM, open a project, no auth errors in browser console.
* Pre-push hooks pass.

Task 2 — Meeting notes badge in the project hub sidebar

Branch: `manny/meeting-notes-badge` | `page.jsx` + `ProjectHubClient.jsx` | ~1 hour

The project sidebar already has a working `badges` prop — passing `{{ meetings: 3 }}` renders a blue dot (collapsed sidebar) or a numbered pill (expanded sidebar) on the Meetings tab. Nothing passes it yet. This task wires up the count.

How badges already works (read-only — do not edit this file)

`src/components/project/ProjectSidebar.jsx` line 22:

```
export default function ProjectSidebar({ config, activeTab, onTabChange, userId, badges = {} }) {
  // ...
  const badge = badges[item.id];  // item.id 'meetings' -> meetings tab badge
  // Collapsed: renders a blue dot when badge > 0
  // Expanded:  renders a blue number pill when badge > 0
}
```

Step 1 — add notes count in page.jsx

Find `src/app/(app)/project/[id]/page.jsx`. The project is fetched with `prisma.project.findUnique({{ include: {{ ... }} }})`. Add `_count` inside the `include` block:

```
include: {
  _count: {
    select: { notes: { where: { noteType: 'PROJECT_MEETING' } } },
  },
  projectData: true,
  responses: true,
  // ... rest unchanged
},
```

Then pass the count to `ProjectHubClient` as a new prop:

```
<ProjectHubClient
  {/* ... existing props unchanged ... */}
  meetingNotesCount={project._count?.notes ?? 0}
/>
```

Step 2 — accept prop and pass to ProjectSidebar in ProjectHubClient.jsx

Find the function signature at the top of `ProjectHubClient` and add the new prop:

```
export default function ProjectHubClient({ project, ..., meetingNotesCount = 0 }) {
```

Then find `<ProjectSidebar ... />` in the JSX and add the `badges` prop:

```
<ProjectSidebar
  config={navConfig}
  activeTab={activeTab}
  onTabChange={setActiveTab}
  userId={userId}
  badges={{ meetings: meetingNotesCount }}
/>
```

That is the entire change. Do not edit `ProjectSidebar.jsx`.

Acceptance criteria

* Open a project that has meeting notes — the Meetings tab shows a blue dot (collapsed) or a blue number (expanded).
* Projects with zero meeting notes show nothing extra on the tab.
* Pre-push hooks pass.

Task 3 — Upcoming deadlines page

Branch: `manny/crm-deadlines` | 2 new files + `CRMClient.jsx` | ~3-4 hours

Projects have date fields in the DB — expected order dates, construction start, MEP rough-in, construction end — that appear nowhere in the app. This page surfaces all of them for open projects so Mike can see what's coming up without opening each project individually.

Model this on the existing Brief page

Before writing a line, read `src/app/(app)/crm/brief/page.jsx` and `BriefClient.jsx` in full — the structure is identical to what you're building.

Step 1 — create src/app/(app)/crm/deadlines/page.jsx

```
export const dynamic = 'force-dynamic';  // REQUIRED
import { redirect } from 'next/navigation';
import { getSession } from '@/lib/auth';
import { prisma } from '@/lib/db';
import DeadlinesClient from './DeadlinesClient';

export default async function DeadlinesPage() {
  const session = await getSession();
  if (!session) redirect('/login');

  const projects = await prisma.project.findMany({
    where: {
      OR: [{ procurementStatus: 'OPEN' }, { designStatus: 'OPEN' }],
    },
    select: {
      id: true, name: true, projectNumber: true,
      procurementStatus: true, designStatus: true,
      procurementStage: true, designStage: true,
      expectedOrderDate: true,
      constructionStartDate: true,
      mepRoughInDate: true,
      constructionEndDate: true,
      client: { select: { name: true } },
    },
    orderBy: { name: 'asc' },
  });

  return <DeadlinesClient projects={projects} />;
}
```

Step 2 — create src/app/(app)/crm/deadlines/DeadlinesClient.jsx

Add `'use client';` at the top.

Date utilities:

```
function parseDate(val) {
  if (!val) return null;
  const d = new Date(val);
  return isNaN(d.getTime()) ? null : d;
}
```

```
function fmtDate(val) {
  const d = parseDate(val);
  if (!d) return null;
  return d.toLocaleDateString('en-US', { month: 'short', day: 'numeric', year: 'numeric' });
}

// Returns: 'overdue' | 'soon' (<=30d) | 'upcoming' (31-90d) | 'later' | null
function dateStatus(val) {
  const d = parseDate(val);
  if (!d) return null;
  const days = Math.floor((d - Date.now()) / 86_400_000);
  if (days < 0)   return 'overdue';
  if (days <= 30) return 'soon';
  if (days <= 90) return 'upcoming';
  return 'later';
}

const STATUS_COLORS = {
  overdue:  'text-red-600 font-semibold',
  soon:     'text-amber-600 font-semibold',
  upcoming: 'text-green-700',
  later:    'text-slate-500',
};
```

Display structure:

Render two sections. Filter out projects with no dates set for each section. If a section is empty, show a `text-slate-400` message.

Section	Filter	Columns
Expected Order Dates	projects where expectedOrderDate is not null	Project name (link to /project/[id]), Client, Stage, Expected Order (co
Construction Milestones	projects where any construction date is not null	Project name (link), Client, Const. Start (colored), MEP Rough-In (co

For date cells: wrap each date in a `<span className={{STATUS_COLORS[dateStatus(val)] ?? ''}}>`. Show – for null dates.

Add a `Back to CRM` link at the top (`href="/crm"`). Use the same header/table style as `BriefClient.jsx`.

Step 3 — add 'Deadlines' link in CRMClient.jsx

Find the existing `Brief` link in the CRM toolbar (search for `href="/crm/brief"`). Add a matching `Deadlines` link right next to it:

```
<Link
  href="/crm/deadlines"
  className="text-xs text-slate-500 hover:text-amber-600 font-medium transition-colors"
>
  Deadlines &rarr;
</Link>
```

Acceptance criteria

* `/crm/deadlines` loads without errors. `force-dynamic` present. Auth gates the page.
* Two sections render. Dates are colored: red = overdue, amber = within 30 days, green = 31-90 days, grey = 90+ days.
* Empty sections show a polite message instead of an empty table.
* 'Deadlines' link appears in the CRM header and navigates correctly.
* Pre-push hooks pass.

Operating reminder (for Claude): Tell Manny what to do — not how to do it technically. He approves proposed changes; Claude handles git, tests, and pushes. Manny never pushes to main — always via a manny/* branch and PR to Sean. If pre-push hooks fail, diagnose and fix. Never --no-verify.